# Multer in Express.js

Quick, simple notes — definitions, setup, and key concepts

---

## 1. What is Multer?

Multer is a Node.js middleware used with Express to handle **multipart/form-data** — the encoding type used when a form uploads files (images, PDFs, videos, etc). Express alone cannot read file uploads; Multer parses the incoming file data and makes it available on the request object.

> It does **not** handle data that is not multipart (like plain JSON or urlencoded forms) — for that you still use `express.json()` or `express.urlencoded()`.

## 2. Installation

```
npm install multer
```

## 3. Basic Setup

Import multer and create an **upload** instance. This instance is the middleware you attach to routes.

```
const express = require('express');
const multer = require('multer');
const app = express();

const upload = multer({ dest: 'uploads/' });
// dest = folder where uploaded files are saved
```

## 4. Storage Engines

Multer gives two ways to control how and where files are stored:

### a) DiskStorage — save files to disk with custom name/path

```
const storage = multer.diskStorage({
  destination: function (req, file, cb) {
    cb(null, 'uploads/');           // folder to save in
  },
  filename: function (req, file, cb) {
    cb(null, Date.now() + '-' + file.originalname); // custom file name
  }
});

const upload = multer({ storage: storage });
```

**cb(error, value)** — the callback pattern used in both functions. First argument is an error (null if none), second is the actual value.

### b) MemoryStorage — keep file in memory as a Buffer

```
const storage = multer.memoryStorage();
const upload = multer({ storage: storage });
// file is available as req.file.buffer (not saved to disk)
// useful when uploading directly to cloud storage (S3, Cloudinary, etc.)
```

## 5. Handling Uploads in Routes

Multer gives different methods depending on how many files you expect:

| Method | Use case | Access in route |
|---|---|---|
| upload.single('field') | One file | req.file |
| upload.array('field', max) | Multiple files, same field | req.files (array) |
| upload.fields([...]) | Multiple fields, different names | req.files (object) |
| upload.none() | No files, only text fields | req.body only |

### Example — single file upload:

```
app.post('/upload', upload.single('avatar'), (req, res) => {
  console.log(req.file);   // uploaded file info
  console.log(req.body);   // other text fields from the form
  res.send('File uploaded');
});
```

### Example — multiple files (same field name):

```
app.post('/upload-multiple', upload.array('photos', 5), (req, res) => {
  console.log(req.files);  // array of file objects
  res.send('Files uploaded');
});
```

### Example — multiple fields with different names:

```
app.post('/upload-fields', upload.fields([
  { name: 'avatar', maxCount: 1 },
  { name: 'gallery', maxCount: 8 }
]), (req, res) => {
  console.log(req.files.avatar);
  console.log(req.files.gallery);
  res.send('Uploaded');
});
```

## 6. The req.file Object

When a single file is uploaded, Multer attaches these properties to `req.file`:

- **fieldname** — name of the form field
- **originalname** — original file name on the user's device
- **encoding** — encoding type of the file
- **mimetype** — file type, e.g. image/png
- **size** — file size in bytes
- **destination** — folder where file was saved (diskStorage only)
- **filename** — name used to save the file (diskStorage only)
- **path** — full path to the saved file (diskStorage only)
- **buffer** — file data as Buffer (memoryStorage only)

## 7. File Filtering (fileFilter)

Use `fileFilter` to accept or reject files, e.g. only allow images.

```
const upload = multer({
  storage: storage,
  fileFilter: function (req, file, cb) {
    if (file.mimetype === 'image/png' || file.mimetype === 'image/jpeg') {
      cb(null, true);   // accept file
    } else {
      cb(new Error('Only PNG/JPEG allowed'), false); // reject file
    }
  }
});
```

## 8. Setting Limits

Control file size, number of files, field size, etc. to prevent abuse.

```
const upload = multer({
  storage: storage,
  limits: {
    fileSize: 2 * 1024 * 1024,  // 2MB max per file
    files: 5                    // max 5 files
  }
});
```

## 9. Error Handling

Multer throws a `MulterError` for issues like file too large or too many files. Catch it with an error-handling middleware.

```
app.post('/upload', (req, res) => {
  upload.single('avatar')(req, res, function (err) {
    if (err instanceof multer.MulterError) {
      return res.status(400).json({ error: err.message });
    } else if (err) {
      return res.status(500).json({ error: 'Unknown error' });
    }
    res.send('Uploaded successfully');
  });
});
```

## 10. Key Points to Remember

- Multer only processes **multipart/form-data** requests.
- Always attach Multer middleware **before** the route handler that needs req.file/req.body.
- Use **diskStorage** to save files on the server, **memoryStorage** to keep them as a buffer (e.g. before uploading to cloud storage).
- **fileFilter** controls which files are accepted.
- **limits** protects your server from very large or too many uploads.
- req.body works for text fields only if Multer processed the form (even upload.none() is needed for multipart forms with no files).